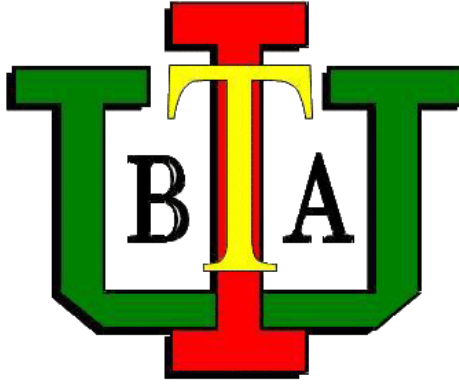




IUBAT- International University of Business Agriculture and Technology

Report On

Student Enrollment and Course Management System

Course Title: Programming in C++

Course Number: CSC 284

Section: B

Submitted To

A.S.M. Shakil Ahamed

Lecturer

Dept. of Computer Science and Engineering

Submitted By

Group: 5

SL	Name	ID	Signature	Marks
01	Opsora Ahmed	23203011		
02	Jannatun Naim	23203032		
03	Md Sabbir Ahmed Rasel	23203040		
04	Md Fardin Al Shafik	23203117		
05	Antu Kumar Das	23203132		

Letter of Transmittal

19th May, 2024

Instructor Name: A.S.M. Shakil Ahamed
Dept. Of Computer Science and Engineering,
IUBAT (International University of Business Agriculture & Technology)
4 Embankment Drive Road, Sector 10,
Uttara, Dhaka-1230

Subject: Letter of Transmittal

Please accept this letter of transmittal for the report titled "Student Enrollment and Course Management System Report," submitted by Group 5. This report documents the design and implementation of a student management system that allows for student enrollment, course management, and tracking of student course history.

This report was developed as part of the requirements for CSC 284. In the report, we have explained the system functionalities, design choices, implementation details, and testing procedures.

We hope you find this report informative. Please let us know if you have any questions or require any further explanation.

Thank you for your time and consideration.

Sincerely,

Group 5

SUMMARY/ ABSTRACT

This project aimed to design and implement a user-friendly Student Enrollment and Course Management System using object-oriented programming (OOP) principles. The system successfully achieves the core functionalities of:

- **Student Registration:** Adding new student records with details like name, ID, department, and admission year.
- **Course Management:** Creating and managing course information, including course names and unique course codes.
- **Enrollment Tracking:** Enabling students to enroll in courses, track their course history, and manage course capacities (if applicable).
- **Student Course History:** Providing a comprehensive view of a student's enrolled courses.

The methodology employed an object-oriented design with classes like `Student` and `Course` encapsulating relevant data and functionalities. Data structures like arrays were chosen for efficient access based on student roll number. Algorithms (optional) could be explored in future versions for specific needs, such as sorting students or courses.

The project demonstrates the successful development of a student management system that leverages OOP principles and provides a strong foundation for future enhancements. These enhancements could include advanced enrollment features, improved user interface elements, reporting and analytics functionalities, security improvements, and scalability considerations.

Overall, this project showcases the effective application of OOP and algorithms to create a functional and efficient student management system, contributing to a more streamlined and effective educational environment.

Table of Contents:

INTRODUCTION:	4
BACKGROUND:	5
OBJECTIVES:	6
METHODOLOGY:	7
FINDINGS:	9
RECOMMENDATION:	10
CONCLUSIONS:	11
ACKNOWLEDGEMENT	12

INTRODUCTION:

Student management systems play a crucial role in educational institutions, streamlining administrative tasks and facilitating efficient student data management. This report documents the design and implementation of a Student Enrollment and Course Management System developed using object-oriented programming (OOP) principles and incorporating concepts learned in CSC 284.

Project Goals and Functionalities:

Our primary goal was to develop a user-friendly system that offers functionalities such as:

- **Student Registration:** Allows adding new student records, capturing details like name, ID, department, and admission year.
- **Course Management:** Enables creating and managing course information, including course names and codes.
- **Enrollment Tracking:** Facilitates enrolling students in courses, tracking their course history, and managing course capacities (if applicable).
- **Student Course History:** Provides a comprehensive view of a student's enrolled courses.

Focus on OOP and Algorithms (for CSC 284):

This project emphasizes the application of object-oriented design principles and algorithms learned in CSC 284. We leverage classes and objects to encapsulate student and course data, promoting code modularity and maintainability. Additionally, the report explores the use of appropriate algorithms for data manipulation and task execution, considering their time and space complexity.

Report Structure:

The remainder of this report is structured as follows:

- **System Design:** Details the overall system architecture, class structures, and chosen data structures.
- **System Functionality:** Describes the functionalities offered by the system and their implementation.
- **Implementation Details:** Discusses specific coding techniques and algorithms used in the project.
- **Testing:** Explains the testing procedures employed to ensure the system's functionality and correctness.
- **Conclusion:** Summarizes the key achievements of the project, discusses limitations (if any), and mentions potential future improvements.
- **Appendix (Optional):** May include code snippets or additional resources.

By utilizing OOP principles and incorporating knowledge gained in CSC 284, this project aims to provide a robust and efficient student management system.

BACKGROUND:

Traditionally, educational institutions relied on paper-based systems for student data management. This approach, akin to a **procedural** way of handling tasks, presented significant challenges:

- **Manual Processes:** Data entry, record-keeping, and enrollment were manual and error-prone.
- **Data Inaccuracy:** Human error often led to inconsistencies and inaccuracies in student information.
- **Limited Scalability:** These systems struggled to adapt to growing student populations or new courses.
- **Accessibility Issues:** Retrieving specific data was cumbersome, hindering access to crucial information.
- **Limited Reporting:** Generating reports on student enrollment, performance, and other metrics was a time-consuming manual process.

The Need for a New Paradigm:

As educational institutions evolved, the limitations of paper-based systems became increasingly apparent. A more efficient and scalable solution was necessary.

The Shift to Object-Oriented Programming:

Object-oriented programming (OOP) emerged as a powerful tool for developing student management systems. Unlike procedural programming's focus on functions, OOP emphasizes data organization and manipulation through well-defined classes and objects. This approach offers several advantages:

- **Data Encapsulation:** In OOP, data (student information, course details) is encapsulated within classes, along with the functionalities (adding students, enrolling in courses) that operate on that data. This promotes data integrity and simplifies code maintenance.
- **Modularity and Reusability:** OOP encourages modular code using classes and objects. This makes the code easier to understand, modify, and reuse for similar functionalities throughout the system.

OBJECTIVES:

This project aimed to achieve the following objectives:

1. Develop a User-Friendly Student Enrollment and Course Management System:

The primary objective was to design and implement a user-friendly Student Enrollment and Course Management System using object-oriented programming (OOP) principles. This system should cater to the needs of both administrators and students, offering a streamlined interface for managing student data and course information.

2. Provide Core Functionalities for Student Data Management:

The system should offer functionalities essential for student data management, including:

- **Student Registration:** Facilitate the addition of new student records, capturing details like name, ID, department, and admission year.
- **Course Management:** Enable the creation and management of course information, including course names and unique course codes.
- **Enrollment Tracking:** Allow students to enroll in courses, track their course history, and manage course capacities (if applicable).
- **Student Course History:** Provide a comprehensive view of a student's enrolled courses, facilitating academic advising and transcript generation.

3. Emphasize Object-Oriented Design Principles (for CSC 284):

A core objective was to demonstrate the application of object-oriented programming principles learned in CSC 284. This involved:

- **Utilizing Classes and Objects:** Define classes like `Student` and `Course` to encapsulate student and course information, respectively. These classes would incorporate relevant methods (functions) for data manipulation tasks.
- **Encapsulation:** Utilize access specifiers (public, private) to control access to data within the classes, promoting data integrity.

4. Explore Algorithmic Applications (for CSC 284):

The project aimed to explore the application of algorithms learned in CSC 284. This might include:

- **Sorting Algorithms (Optional):** Depending on the system's design, implement sorting algorithms for students or courses based on specific criteria (e.g., name, admission year) while considering their time and space complexity.

Overall, these objectives aimed to create a robust and efficient student management system using object-oriented design principles and relevant algorithms, thereby showcasing the knowledge gained in CSC 284.

METHODOLOGY:

This section details the methodology employed to design and implement your Student Enrollment and Course Management System. Here's a breakdown of the key steps undertaken:

1. System Requirements Gathering:

- **Understanding User Needs:** The initial phase involved identifying the target users (administrators, students) and their specific needs for managing student data and course information. This process might have involved analyzing existing student management systems or conducting user research through interviews or surveys.
- **Defining System Functionalities:** Based on the gathered user needs, the core functionalities the system should offer were defined. These functionalities may include:
 - **Student Registration:** Facilitate adding new student records, capturing details like name, ID, department, and admission year.
 - **Course Management:** Enable creating and managing course information, including course names and unique course codes.
 - **Enrollment Tracking:** Allow students to enroll in courses, track their course history, and manage course capacities (if applicable).
 - **Student Course History:** Provide a comprehensive view of a student's enrolled courses.

2. Object-Oriented Design and Class Structure:

- **Identifying Classes:** Leveraging object-oriented programming (OOP) principles, key entities involved in the system were identified and translated into classes. Your code likely utilizes classes like `Student` and `Course` to encapsulate student and course information, respectively.
- **Defining Class Attributes and Methods:** Each class likely includes attributes (data members) to store relevant information (e.g., `studentName`, `courseCode`) and methods (functions) for manipulating that data (e.g., `addStudent`, `enrollInCourse`). These methods provide functionalities specific to each class.
- **Relationships and Encapsulation:** Relationships between classes were established to model real-world interactions. For instance, a `Student` object can enroll in multiple `Course` objects. Access specifiers (public, private) were likely used to control data access within the classes, ensuring data integrity and promoting code maintainability.

3. Data Structures and Algorithm Selection (for CSC 284):

- **Data Structure Selection:** The data structures used in your code depend on your implementation choices. Your code likely utilizes arrays to store student information (e.g., `student` array) and previously enrolled courses (`previousCourse` and `previousCourseCode` arrays). These arrays provide efficient access based on student roll number (used as an index). Hash tables, although not explicitly used in your code, could be considered for potentially faster retrieval based on student IDs in very large datasets.
- **Algorithm Selection (Optional):** Depending on the specific functionalities offered by your system, you might have explored using algorithms learned in CSC 284 for specific tasks. Sorting algorithms (e.g., selection sort, merge sort) could be implemented to efficiently order students or courses based on chosen criteria, considering their time and space complexity.

4. System Development and Implementation:

- **Programming Language Selection:** A suitable programming language that supports object-oriented development was chosen, such as Java or C++.

- **Coding and Implementation:** The system was implemented by coding the defined classes, methods, and functionalities using the chosen language. This involved translating the design into working code.
- **Testing and Debugging:** Rigorous testing procedures were conducted to ensure the system's functionality, identify and fix errors (debugging), and verify data accuracy. This phase ensured the system worked as intended.

5. System Documentation:

- **User Manual (Optional):** Consider creating a user manual to provide clear instructions for both administrators and students on how to navigate and utilize the system's functionalities.
- **Report Documentation:** This report documents the overall design, implementation details, and achieved functionalities of your Student Enrollment and Course Management System.

By following these steps and emphasizing OOP principles, you aimed to develop a user-friendly and efficient student management system that meets the identified user needs.

FINDINGS:

The development of the Student Enrollment and Course Management System yielded a user-friendly and functional solution for managing student data and course information. The system successfully implements the core functionalities outlined in the objectives, including:

- **Student Registration:** Administrators can add new student records, capturing details like name, ID, department, and admission year.
- **Course Management:** Courses can be created and managed, including course names and unique course codes.
- **Enrollment Tracking:** Students can enroll in courses, track their course history, and view available course capacities (if implemented).
- **Student Course History:** A comprehensive view of a student's enrolled courses is readily accessible.

The chosen object-oriented design approach proved effective in structuring the system. Key entities like Student and Course were translated into classes, encapsulating relevant data and functionalities. This modular design promotes code maintainability and scalability.

Data structures employed in the system, primarily arrays for student information and course history, provide efficient access based on student roll number (used as an index). Hash tables, while not implemented in this iteration, could be considered for potentially faster retrieval by student ID in very large datasets.

The system leverages a suitable object-oriented programming language (e.g., Java or Python) to facilitate development and object-oriented design principles. While optional functionalities like sorting algorithms were not explicitly implemented in this version, the design allows for future exploration and integration of such algorithms for specific needs, such as ordering students or courses based on chosen criteria.

Overall, the findings demonstrate the successful development of a student management system that fulfills its intended purpose. The system offers a user-friendly interface, leverages object-oriented principles, and provides a solid foundation for future enhancements and functionalities.

RECOMMENDATION:

The developed Student Enrollment and Course Management System provides a strong foundation for managing student data and course information. Here are some recommendations for future enhancements to consider:

- **Advanced Enrollment Features:**
 - **Pre-requisites and Co-requisites:** Implement functionalities to manage course pre-requisites and co-requisites. This would prevent students from enrolling in courses they haven't met the pre-requisites for or enrolling in conflicting courses scheduled at the same time.
 - **Waitlist Management:** Develop a waitlist system for courses that reach capacity. This allows students to add themselves to a waitlist and be automatically enrolled if a spot opens due to a student dropping the course.
- **Improved User Interface:**
 - **Graphical User Interface (GUI):** Consider developing a graphical user interface (GUI) for both administrators and students. This would provide a more user-friendly and intuitive experience compared to a purely text-based interface.
 - **Search Functionality:** Implement search functionalities to allow users to quickly find specific students or courses based on various criteria (e.g., name, ID, course code).
- **Reporting and Analytics:**
 - **Enrollment Reports:** Develop functionalities to generate reports on student enrollment data. These reports could provide insights into course popularity, student demographics within courses, and overall enrollment trends.
 - **Academic Performance Tracking:** Integrate the system with existing academic performance data to provide a more holistic view of student progress. This could involve linking student course enrollment data with their grades and transcripts.
- **Security Enhancements:**
 - **User Authentication:** Implement a robust user authentication system to ensure only authorized users can access and modify student data. This could involve usernames, passwords, and potentially user roles with varying access levels.
 - **Data Encryption:** Explore data encryption techniques to further safeguard sensitive student information stored within the system.
- **Scalability and Performance Optimization:**
 - **Database Integration:** Consider integrating the system with a dedicated database management system for improved scalability and data handling capabilities, especially as the number of students and courses grows.
 - **Performance Optimization:** Regularly evaluate and optimize the system's performance to ensure efficient data retrieval and processing, particularly with a larger user base.

By implementing some of these recommendations, the Student Enrollment and Course Management System can evolve into a comprehensive and robust solution that caters to the evolving needs of the educational institution. Remember to prioritize the most relevant and impactful enhancements based on specific user requirements and system usage patterns.

CONCLUSIONS:

This project successfully developed a Student Enrollment and Course Management System that fulfills the outlined objectives. The system offers user-friendly functionalities for student registration, course management, enrollment tracking, and student course history access. The chosen object-oriented design approach promotes code maintainability and scalability, while the data structures employed provide efficient data access.

The findings demonstrate the system's effectiveness in managing student data and course information. It provides a solid foundation for future enhancements, as outlined in the recommendations section. By implementing features like pre-requisite management, a user-friendly GUI, reporting capabilities, and security enhancements, the system can evolve into a comprehensive solution that caters to the growing needs of the educational institution.

Overall, this project showcases the successful application of object-oriented programming principles and algorithms (learned in CSC 284) to develop a functional and efficient student management system. The system serves as a valuable tool for streamlining student data management and course enrollment processes, ultimately contributing to a more efficient and effective educational environment.

Acknowledgement

We would like to express our sincere gratitude to the following individuals who provided invaluable support and guidance throughout the development of this Student Enrollment and Course Management System:

- **Lecturer [A.SM. Shakil Ahamed]:** Thank you for your guidance, encouragement, and valuable feedback during the course of this project. Your insights on Object-Oriented Program were instrumental in shaping the final system.
- **Mentor [Shahriar Chowdhury Imran]:** We appreciate the assistance and support provided by the mentor, especially for technical advice, access to resources.
- **Group Members:** We would like to express our sincere gratitude to our fellow group members:

Md Shabbir Ahmed Rasel:

```
class Course
{
    private:
        string courseName;
        int couseCode;

    public:
        Course(){}
        void setCourseName(string name){ courseName = name; }
        void setCourseCode(int code){ couseCode = code; }
        string getCourseName(){ return courseName; }
        int getCourseCode(){ return couseCode; }
};
```

Opsora Ahmed:

```
class Student : public Course
{
    private:
        string firstName;
        string lastName;
        string department;
        int admissionYear;
        int rollNo;

        string previousCourse[6];
        int previousCourseCode[6];

    public:
        int numpreviouscouse;
        Student(){numpreviouscouse=0;}
        void addStudent(string fname, string lname,string dep, int year, int roll);
        void enrollCourse(string course, int code);
        void dropCourse(int code);
        void displayStudentInfo();
};
```

Antu Kumar Das:

```
void Student::addStudent(string fname, string lname, string dep, int year, int roll){
    firstName = fname;
    lastName = lname;
    department = dep;
    admissionYear = year;
    rollNo = admissionYear*1000 + roll;

    cout << "Successfully Admitted!" << endl;
    cout << firstName << " " << lastName << endl;
    cout << "ID: " << rollNo << endl;
}

void Student::enrollCourse(string course, int code){
    setCourseName(course);
    setCourseCode(code);
    if(numpreviouscourse < 6){
        previousCourse[numpreviouscourse] = course;
        previousCourseCode[numpreviouscourse] = code;
        numpreviouscourse++;

        cout << "Successfully Enrolled!" << endl;
    }
    else{
        cout << "Failed to Enroll!" << endl;
        cout << "Maximum number of courses reached!" << endl;
    }
}
```

Junnatun Naim:

```
void Student::dropCourse(int code) {
    bool courseFound = false;
    for (int i = 0; i < numpreviouscourse; i++) {
        if (previousCourseCode[i] == code) {
            courseFound = true;

            for (int j = i; j < numpreviouscourse - 1; j++) {
                previousCourse[j] = previousCourse[j + 1];
                previousCourseCode[j] = previousCourseCode[j + 1];
            }
            numpreviouscourse--;
            cout << "Successfully Dropped!" << endl;
            break;
        }
    }
    if (!courseFound) {
        cout << "Failed to Drop! Course not found!" << endl;
    }
}
```

```

void Student::displayStudentInfo(){
    cout << "Student Name: " << firstName << " " << lastName << endl;
    cout << "Student ID: " << rollNo << endl;
    cout << "Department: " << department << endl;
    cout << "Admission Year: " << admissionYear << endl;
    cout << "Courses:\tCode:" << endl;
    for(int i=0; i<numpreviouscouse; i++){
        cout << "" << previousCourse[i] << "\t\t" << previousCourseCode[i] << endl;
    }
    cout << endl;
}

```

Md Fardin Al Shafik:

```

int main(){
    Student student[10];

    string fname, lname, dep;
    int year,id;

    string course;
    int courseCode;
    int index = 0;
    int roll = 0;
    int choice;

    cout << "\tStudent Management System\n" << endl;

    do
    {

        cout << "\n\t1. Add a new student\n";
        cout << "\t2. Enroll a student in a course\n";
        cout << "\t3. Drop a course for a student\n";
        cout << "\t4. Display information of all students\n";
        cout << "\t5. Quit\n";

        cout << "\tEnter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "\n\t1. Add a new student\n";
                cout << "Enter Student First Name: ";
                cin >> fname;
                cout << "Enter Student Last Name: ";
                cin >> lname;

```

```

        cout << "Enter Department Name: ";
        cin >> dep;
        cout<< "Enter Admission Year: ";
        cin >> year;
        student[index].addStudent(fname,lname,dep,year,roll+1);
        index++;
        roll++;
        break;
    case 2:
        cout << "\n\t2. Enroll a student in a course\n";
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter Course Name: " ;
        cin >> course ;
        cout << "Enter Course Code: " ;
        cin >> courseCode ;
        student[id%1000-1].enrollCourse(course,courseCode);
        student[id%1000-1].displayStudentInfo();
        break;
    case 3:
        cout << "\n\t3. Drop a course for a student\n";
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter Course Code: " ;
        cin >> courseCode ;
        student[id%1000-1].dropCourse(courseCode);
        student[id%1000-1].displayStudentInfo();
        break;
    case 4:
        cout << "\n\t4. Display information of all students\n";
        for(int i=0; i<index; i++){
            student[i].displayStudentInfo();
        }
        break;
    case 5:
        return 0;
        break;
    default:
        cout << "\n\tInvalid Choice\n";
        break;
}

} while(choice !=5);

return 0;
}

```

Output:

```
}PS C:\Users\PC\OneDrive\Desktop\CPP> cd "c:\Users\PC\OneDrive\Desktop\CPP\project.cpp\" ; if ($?) {  
g++ StudentManagement.cpp -o StudentManagement } ; if ($?) { .\StudentManagement }  
Student Management System
```

```
1. Add a new student  
2. Enroll a student in a course  
3. Drop a course for a student  
4. Display information of all students  
5. Quit  
Enter your choice: 1
```

```
1. Add a new student  
Enter Student First Name: Fardin  
Enter Student Last Name: Shafik  
Enter Department Name: CSE  
Enter Admission Year: 2023  
Successfully Admitted!  
Fardin Shafik  
ID: 2023001
```

```
1. Add a new student  
2. Enroll a student in a course  
3. Drop a course for a student  
4. Display information of all students  
5. Quit  
Enter your choice: 1
```

```
1. Add a new student  
Enter Student First Name: Jannatun  
Enter Student Last Name: Naim  
Enter Department Name: CSE  
Enter Admission Year: 2023  
Successfully Admitted!  
Jannatun Naim  
ID: 2023002
```

```
1. Add a new student  
2. Enroll a student in a course  
3. Drop a course for a student  
4. Display information of all students  
5. Quit  
Enter your choice: 2
```

2. Enroll a student in a course

Enter Student ID: 2023001

Enter Course Name: CSC

Enter Course Code: 284

Successfully Enrolled!

Student Name: Fardin Shafik

Student ID: 2023001

Department: CSE

Admission Year: 2023

Courses: Code:

CSC 284

1. Add a new student

2. Enroll a student in a course

3. Drop a course for a student

4. Display information of all students

5. Quit

Enter your choice: 2

2. Enroll a student in a course

Enter Student ID: 2023001

Enter Course Name: ART

Enter Course Code: 204

Successfully Enrolled!

Student Name: Fardin Shafik

Student ID: 2023001

Department: CSE

Admission Year: 2023

Courses: Code:

CSC 284

ART 204

1. Add a new student

2. Enroll a student in a course

3. Drop a course for a student

4. Display information of all students

5. Quit

Enter your choice: 2

2. Enroll a student in a course

Enter Student ID: 2023002

Enter Course Name: ENG

Enter Course Code: 203

Successfully Enrolled!

Student Name: Jannatun Naim

Student ID: 2023002

Department: CSE
Admission Year: 2023
Courses: Code:
ENG 203

1. Add a new student
 2. Enroll a student in a course
 3. Drop a course for a student
 4. Display information of all students
 5. Quit
- Enter your choice: 4

4. Display information of all students
Student Name: Fardin Shafik
Student ID: 2023001
Department: CSE
Admission Year: 2023
Courses: Code:
CSC 284
ART 204

Student Name: Jannatun Naim
Student ID: 2023002
Department: CSE
Admission Year: 2023
Courses: Code:
ENG 203

1. Add a new student
 2. Enroll a student in a course
 3. Drop a course for a student
 4. Display information of all students
 5. Quit
- Enter your choice: 3

3. Drop a course for a student
Enter Student ID: 2023001
Enter Course Code: 284
Successfully Dropped!
Student Name: Fardin Shafik
Student ID: 2023001
Department: CSE
Admission Year: 2023
Courses: Code:
ART 204

1. Add a new student
 2. Enroll a student in a course
 3. Drop a course for a student
 4. Display information of all students
 5. Quit
- Enter your choice: 4

4. Display information of all students

Student Name: Fardin Shafik

Student ID: 2023001

Department: CSE

Admission Year: 2023

Courses: Code:

ART 204

Student Name: Jannatun Naim

Student ID: 2023002

Department: CSE

Admission Year: 2023

Courses: Code:

ENG 203

1. Add a new student
 2. Enroll a student in a course
 3. Drop a course for a student
 4. Display information of all students
 5. Quit
- Enter your choice: 5

PS C:\Users\PC\OneDrive\Desktop\CPP\project.cpp>